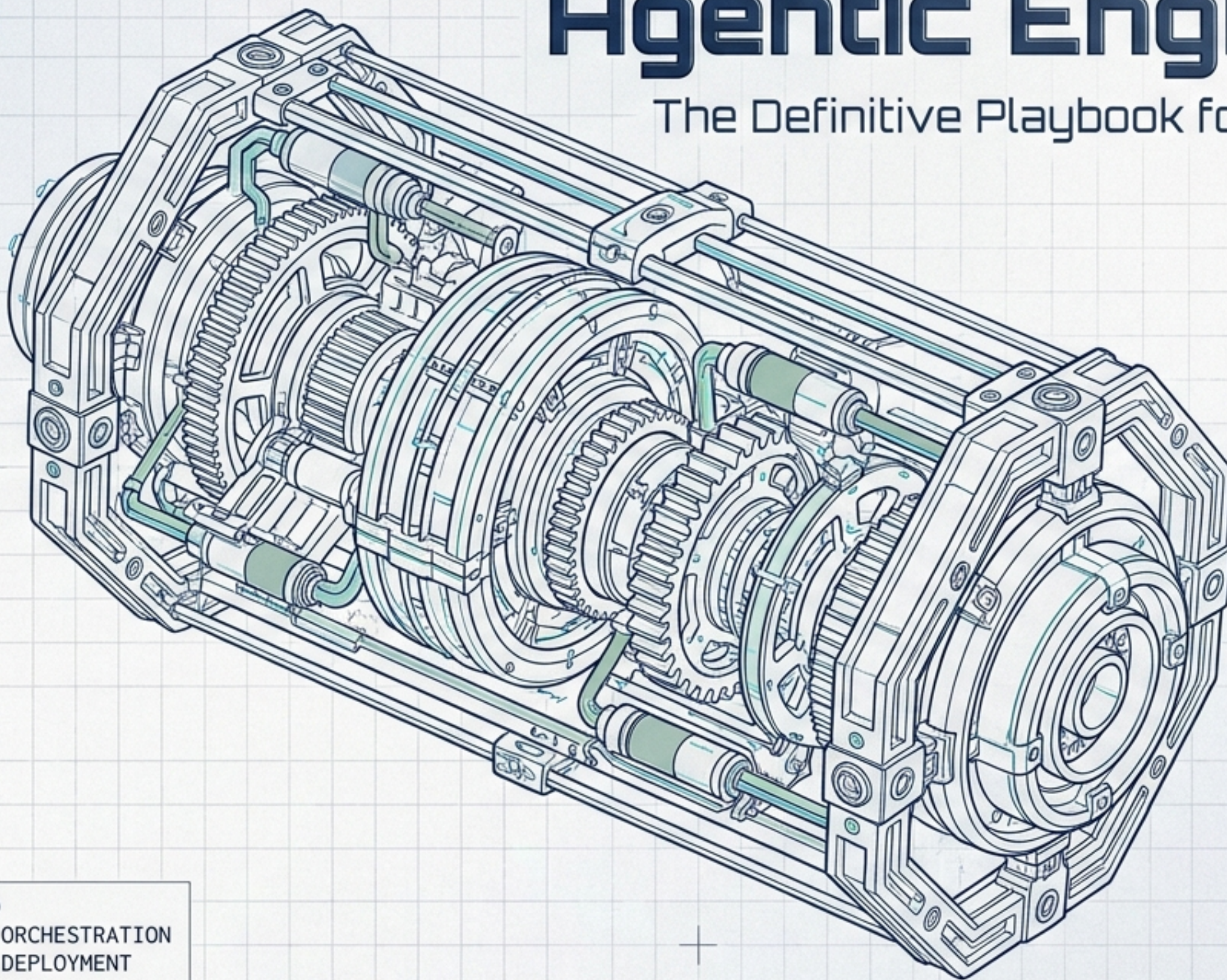


Agentic Engineering

The Definitive Playbook for the Modern SDLC

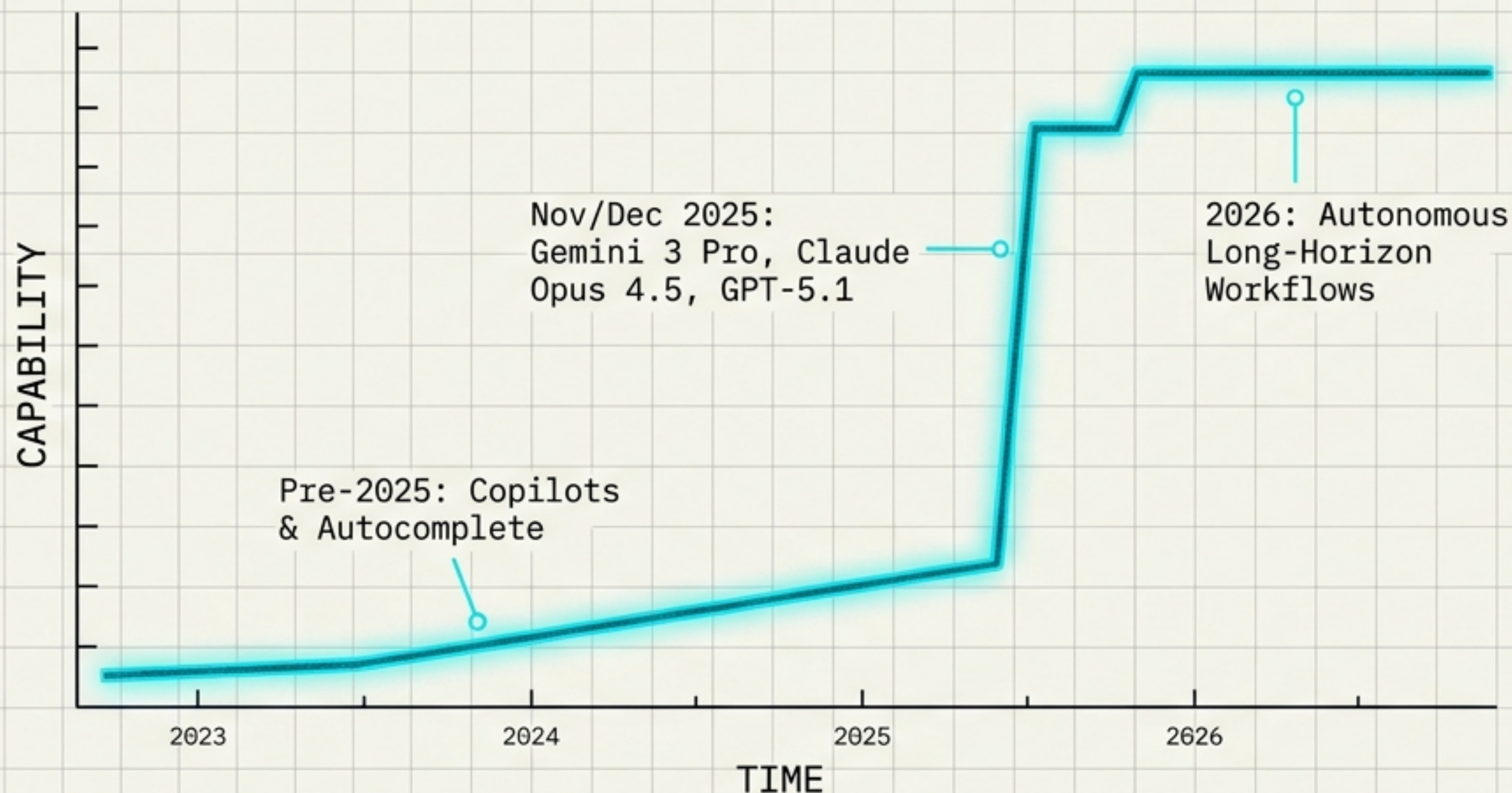


ERA: SOFTWARE 3.0

FOCUS: MULTI-AGENT ORCHESTRATION

TARGET: ENTERPRISE DEPLOYMENT

The December 2025 inflection broke the prototype wall



“I have never felt more **behind as a programmer.** The unit of programming changed from typing lines of code to delegating macro actions.”

— Andrej Karpathy

The default workflow has fundamentally shifted. LLMs are no longer just predictive text generators; they are programmable interpreters executing over digital context.

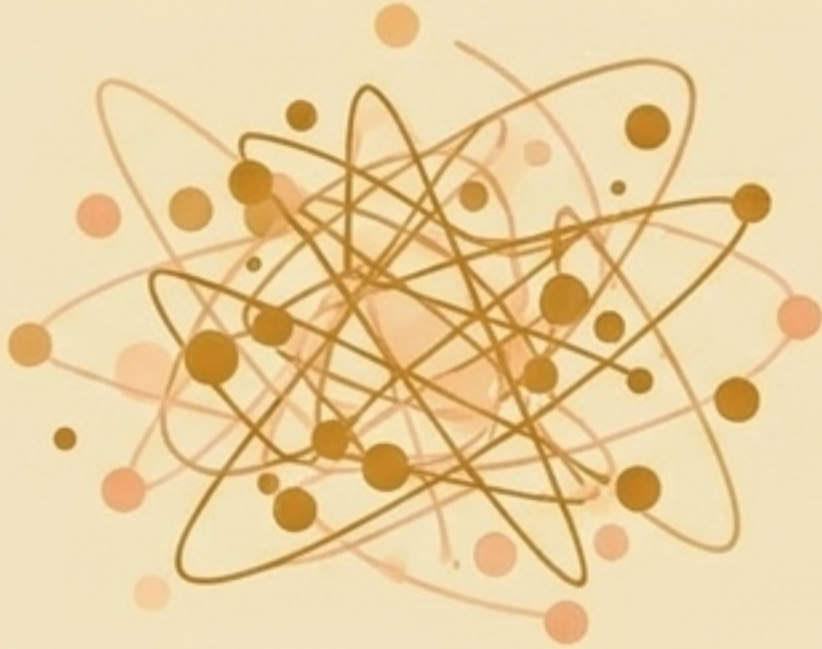
The taxonomy of the modern software development lifecycle

	Pre-2023	2023-24	2025	2026 (Agentic Engineering)
Primary Interface	IDE / Syntax	Autocomplete	Chat / Prompt	Agent Orchestrator
Human Role	Implementation	Implementation	Directing Intent	System Design & Verification
Unit of Work	Functions / Classes	Snippets	Prototypes	Entire Workflows
Source of Truth	Source Code	Source Code	The Prompt	The Trace Log



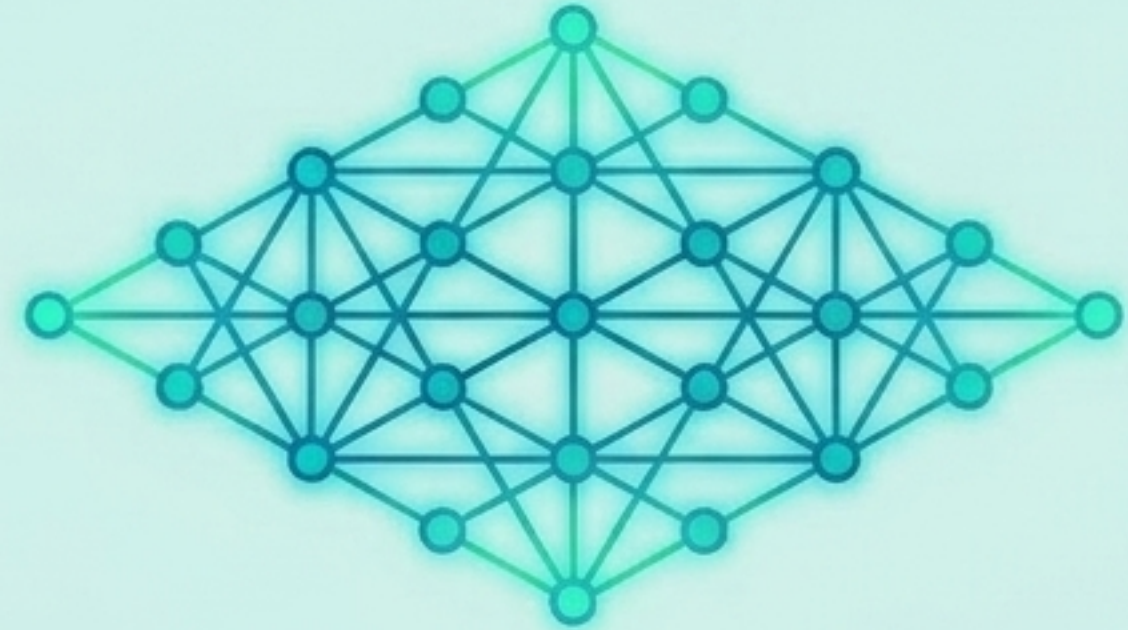
Takeaway: The human role has elevated from manual syntax generation to high-level architectural orchestration.

Vibe Coding raises the floor; Agentic Engineering raises the ceiling



Vibe Coding

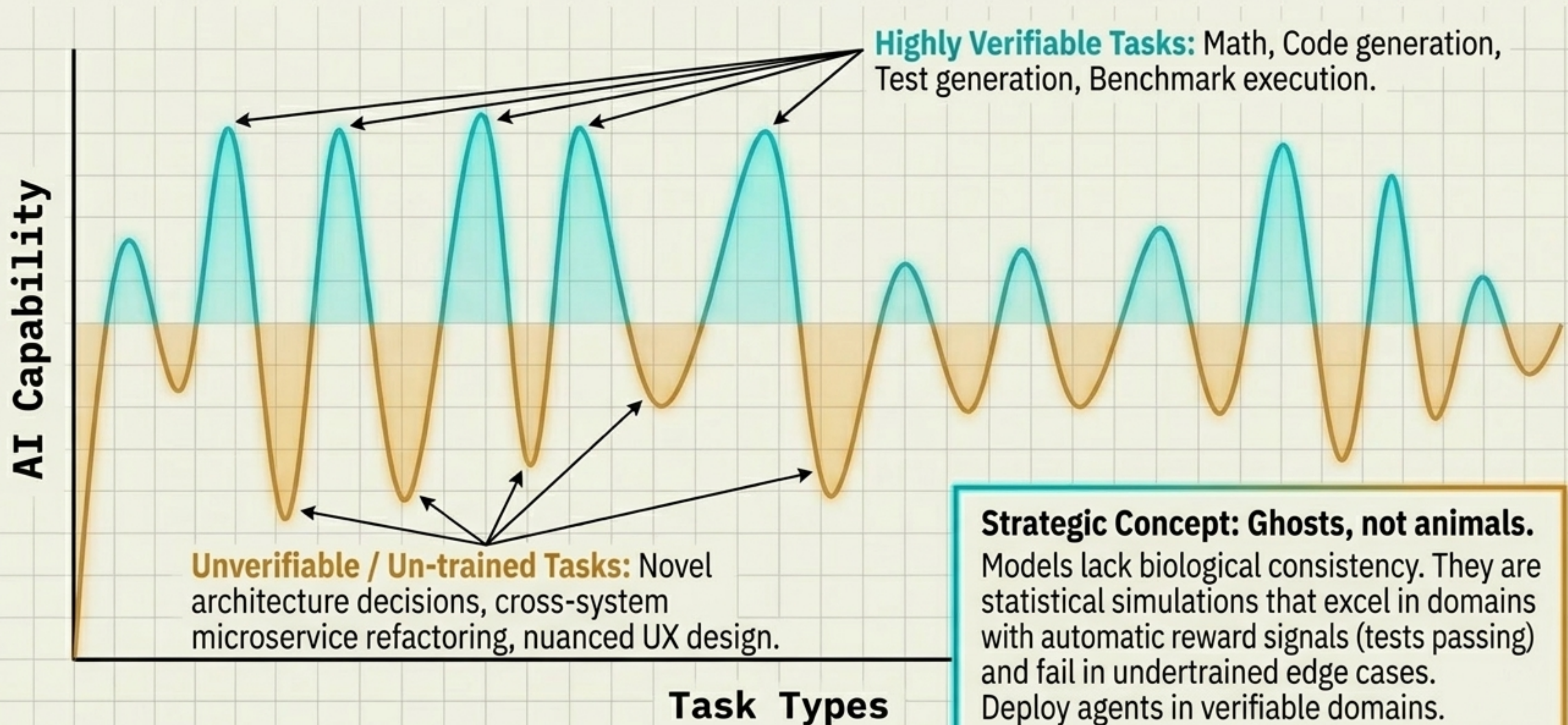
- **Nature:** Intuition-driven, prompt-and-hope.
- **Best for:** Prototypes, one-off scripts, personal tools.
- **Failure Mode:** AI Slop — Code that looks reasonable but introduces security flaws, unmaintainable architecture, and compounding technical debt.



Agentic Engineering

- **Nature:** Structured, governed, deterministic loops.
- **Best for:** Production systems, team codebases, secure environments.
- **Success Mode:** A disciplined methodology where agents plan, write, test, and ship under strict human oversight and automated quality gates.

Navigating the jagged capability curve of frontier models



The harness is the true differentiator, not the base model

Model Context Protocol (MCP)

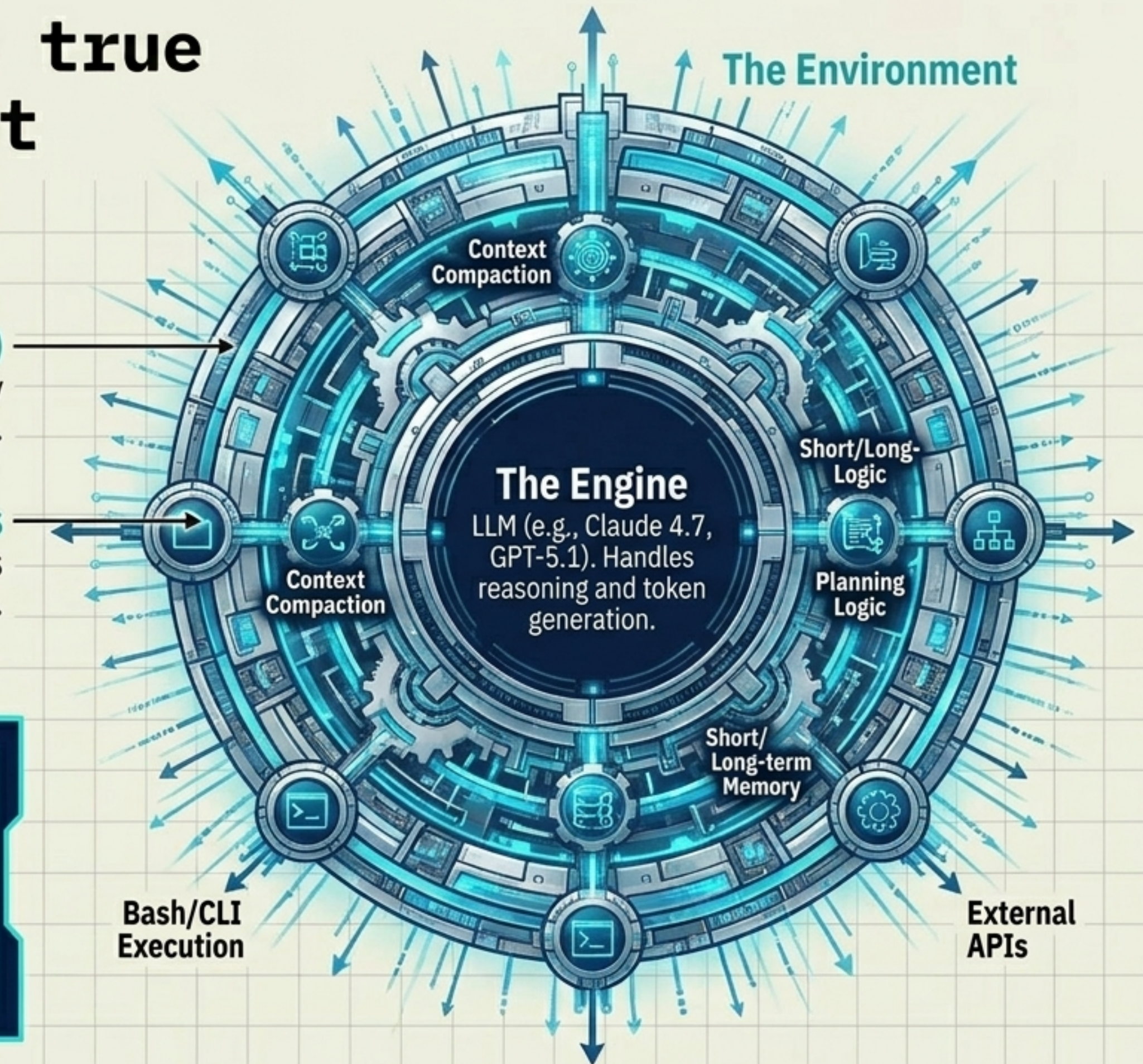
Model, test atomilaris, logit, camrery documenters and nollation.

The Harness

LLM (e.g., Claude 4.7, GPT-5.1). Handles reasoning and token generation.

A raw model is merely a brain in a jar.

The harness provides the hands, eyes, and memory required to execute long-horizon tasks autonomously.



The logic no longer lives entirely in the code

Traditional Software

Source of Truth: Source Code.

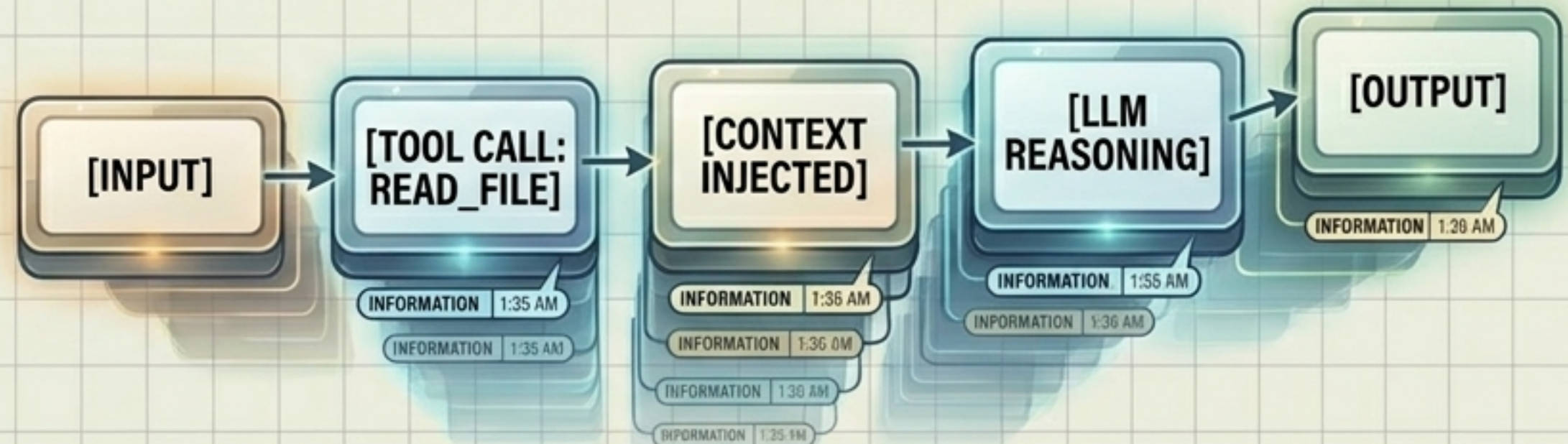
Nature: Deterministic. To know what the app does, you read the syntax.

```
1 function main() {  
2     const data = fetch_data();  
3     if (data.isValid) {  
4         process(data);  
5     }  
6 }
```

Agentic Software

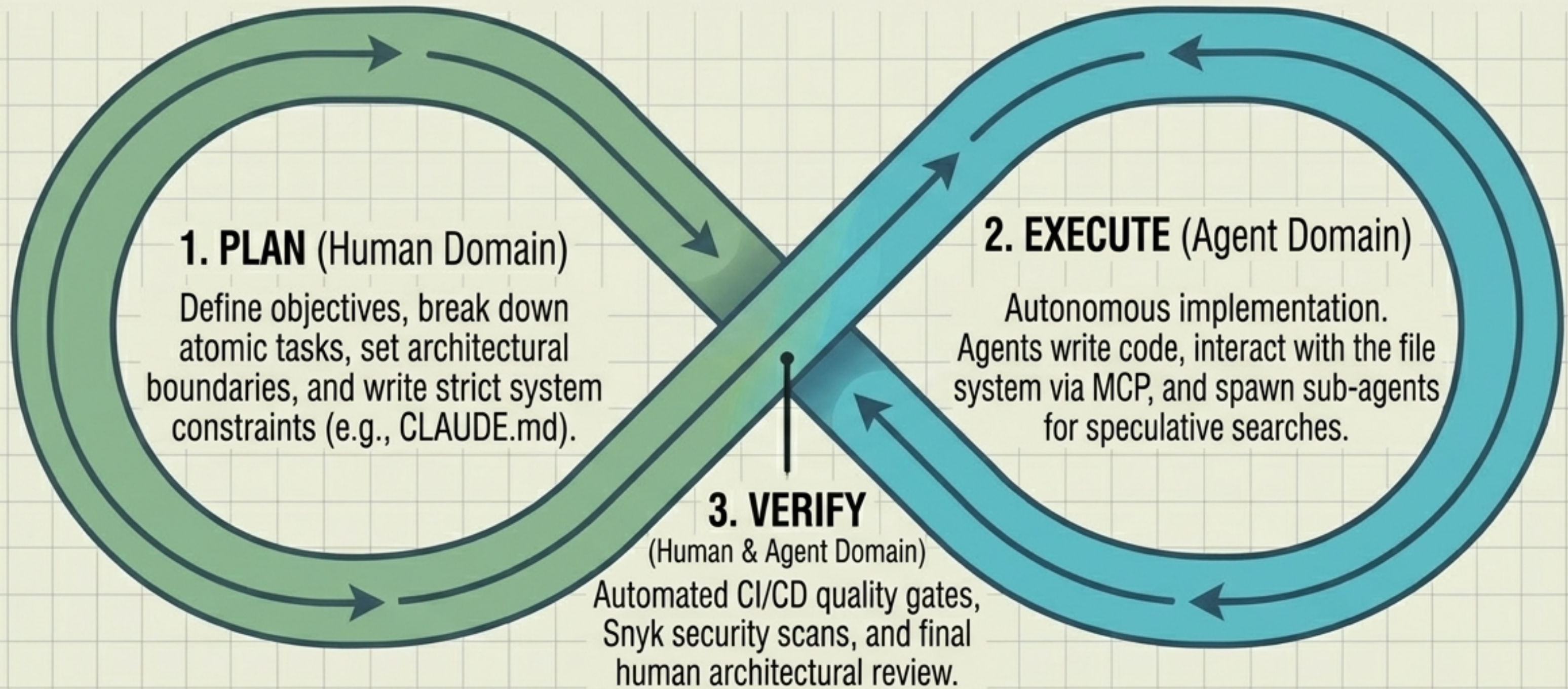
Source of Truth: The Trace Log.

Nature: Probabilistic. Behavior emerges from the model interacting with dynamic context.



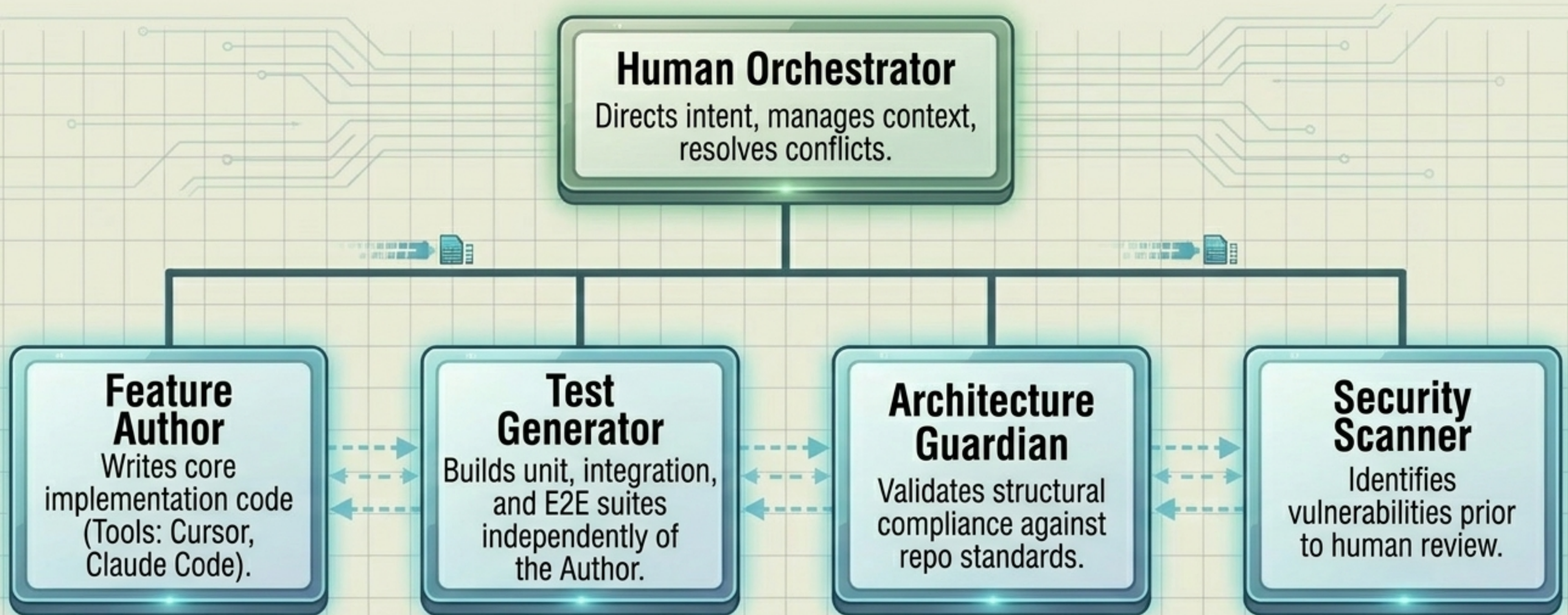
Technical Insight: When an agent fails at Step 14, reading the code won't help. Engineers must inspect the Trace to see exactly what context was injected at Step 13. "Send me a trace" has replaced "Show me the code."

The PEV Loop replaces prompt-and-hope mechanics



Summary: The engineering rigor is front-loaded in the Plan phase and back-loaded in the Verify phase. The execution in the middle is fully automated.

Separation of duties via multi-agent crews



Key Concept: Do not use one monolithic prompt. Assigning specialized roles, goals, and backstories to separate sub-agents prevents context pollution and increases reliability.

The recalibration of the engineering skill stack

Legacy Skills		Agentic Skills
Writing Code		Specifying Intent Precisely
Debugging Code		Debugging Agent Behavior (Traces)
Project Management		Multi-Agent Orchestration
Managing Tech Debt		Managing Cognitive Debt (The cost of poorly managed AI interactions and context loss)

What Stays the Same: Domain expertise, systems thinking, and security awareness. The agent executes, but the human must still know what 'good' software looks like.

2026 Prompting: Steering through structured context

Effort Parameters

Setting 'effort=max' to unlock full 64k token thinking budgets for complex tasks.

Subagent Constraints

Explicit directives (e.g., 'Do not spawn a subagent for work you can complete directly').

PROMPT_PLANNING.xml

```
1 <prompt version="2.0" effort="max">
2   <!-- Main directive -->
3   <directive>Do not spawn a subagent for work you can complete directly.</directive>
4
5   <context_compaction files=".cursorrules, CLAUDE.md">
6     <!-- Persistent guidance -->
7   </context_compaction>
8
9   <planning_process>
10     <step>Analyze the user request against domain constraints.</step>
11   </planning_process>
12
13   <rules>
14     <rule id="R1">Anchor reasoning to provided architecture.</rule>
15     <rule id="R2">Generate code only after validation.</rule>
16   </rules>
17
18   <output_structure>
19     <!-- Expected response format -->
20   </output_structure>
21 </prompt>
```

XML Tagging

Using <planning_process> and <rules> tags to force models to anchor their reasoning before generating output.

Context Compaction

Utilizing .cursorrules and CLAUDE.md to persist durable guidance without filling the active context window with chat history.

The security imperative and machine-speed technical debt

DATA: CORTEX 2026 BENCHMARK

Pull Requests
per author

+20%

Incidents per
Pull Request

+23.5%

The Problem: Machine-Speed Technical Debt

Agents prioritize functioning code over secure, maintainable architecture. An agent writing 1,000 PRs/week with a 1% vulnerability rate creates 10 new security flaws weekly.

The Solution: Automated Defense

Security scanning (Snyk/Semgrep) must be natively embedded into the agentic harness before human review is ever requested.

Enterprise scale: The economics of autonomy

TELUS

500,000 Hours Saved

Deployed 13,000+ custom AI solutions, accelerating engineering velocity by 30%.

Stripe

1,000+ PRs/Week

Internal 'Minions' system fully automates code generation, CI passing, and PR opening with zero human interaction mid-loop.

OpenAI

1 Million Lines of Code

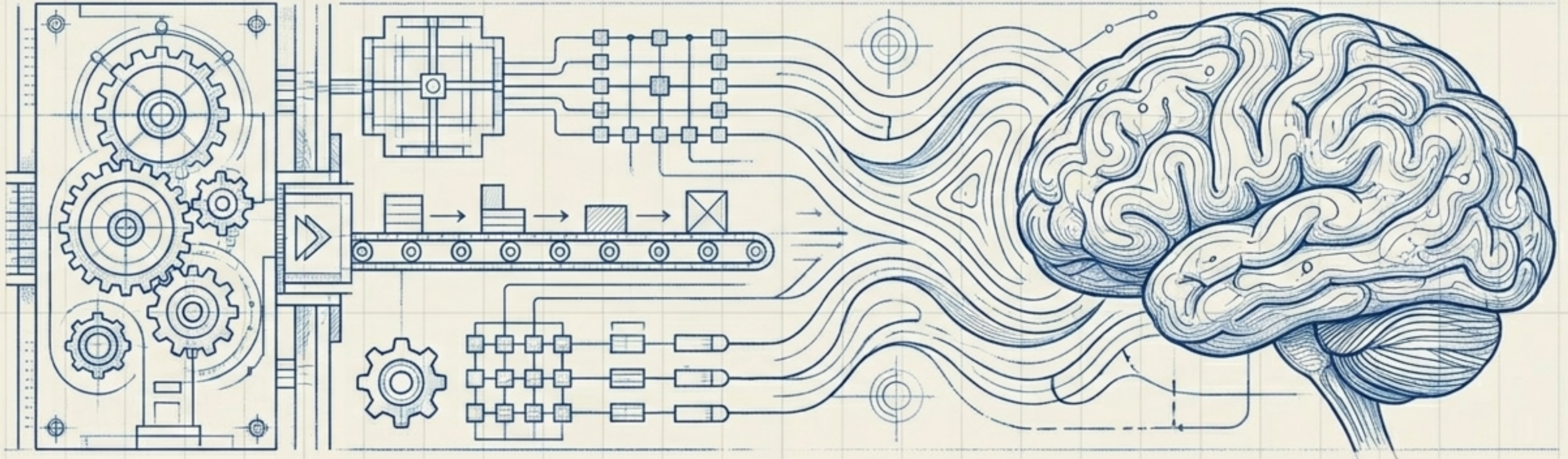
A production application built with zero manually written lines in 1/10th the traditional development time.

Rakuten

12.5M Line Codebase

Complex vLLM implementation completed across massive architecture in just 7 hours with 99.9% accuracy.

You can outsource your thinking, but not your understanding



Automated (The Assembly Line)

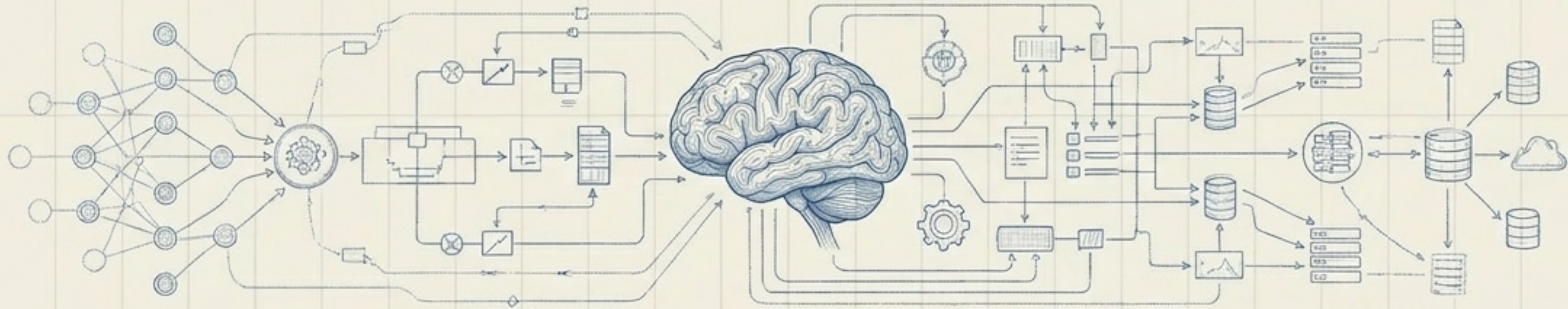
Boilerplate generation, API recall, first drafts, repetitive setup, simple syntax transformations.

Irreplaceable (The Human Brain)

Taste, architectural boundaries, evaluation design, security posture, and recognizing when a model has hallucinated.

Core Insight: While the execution of software development is now automated at scale, the deep understanding required to direct those agents is the ultimate, un-automatable bottleneck.

From crafting code to orchestrating systems



01

Build Harnesses, Not Just Apps

Invest in the infrastructural context and tools that allow agents to operate reliably.

02

Verify at Machine Speed

Replace manual testing with automated, agentic quality gates.

03

Govern the Autonomy

Treat AI agents not as magic tools, but as fallible, powerful software components requiring strict architectural boundaries.

The teams that master the Agentic SDLC will not just ship code faster; they will fundamentally redefine what software can achieve.